9/15 Lab1

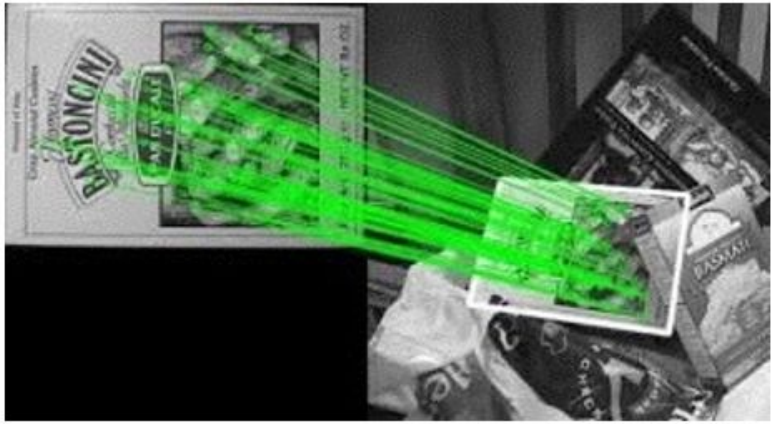
Outline

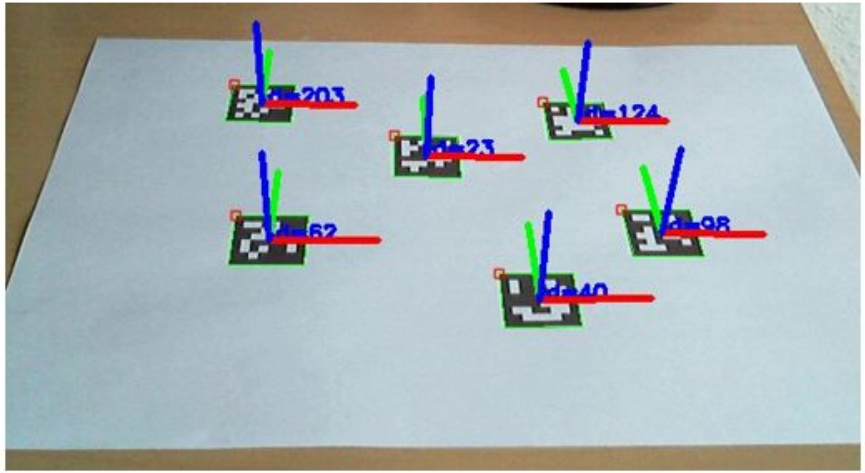
- OpenCV Introduction
- Python 3x & OpenCV installation
- Lab1



- core. The Core Functionality
- imgproc. Image Processing
- imgcodecs. Image file reading and writing
- videoio. Media I/O
- highgui. High-level GUI and Media I/O
- video. Video Analysis
- calib3d. Camera Calibration and 3D Reconstruction
- features2d. 2D Features Framework
- objdetect. Object Detection
- ml. Machine Learning
- flann. Clustering and Search in Multi-Dimensional Spaces
- photo. Computational Photography
- stitching. Images stitching
- cuda. CUDA-accelerated Computer Vision
- cudaarithm. CUDA-accelerated Operations on Matrices
- cudabgsegm. CUDA-accelerated Background Segmentation
- cudacodec. CUDA-accelerated Video Encoding/Decoding
- cudafeatures2d. CUDA-accelerated Feature Detection and Description
- cudafilters. CUDA-accelerated Image Filtering
- cudaimgproc. CUDA-accelerated Image Processing
- cudaoptflow. CUDA-accelerated Optical Flow
- cudastereo. CUDA-accelerated Stereo Correspondence
- cudawarping. CUDA-accelerated Image Warping
- shape. Shape Distance and Matching
- superres. Super Resolution
- videostab. Video Stabilization
- viz. 3D Visualizer
- bioinspired. Biologically inspired vision models and derivated tools
- cvv. GUI for Interactive Visual Debugging of Computer Vision Programs
- datasets. Framework for working with different datasets
- face. Face Recognition
- Binary descriptors for lines extracted from an image
- optflow. Optical Flow Algorithms
- reg. Image Registration
- rgbd. RGB-Depth Processing
- Saliency API
- surface_matching. Surface Matching

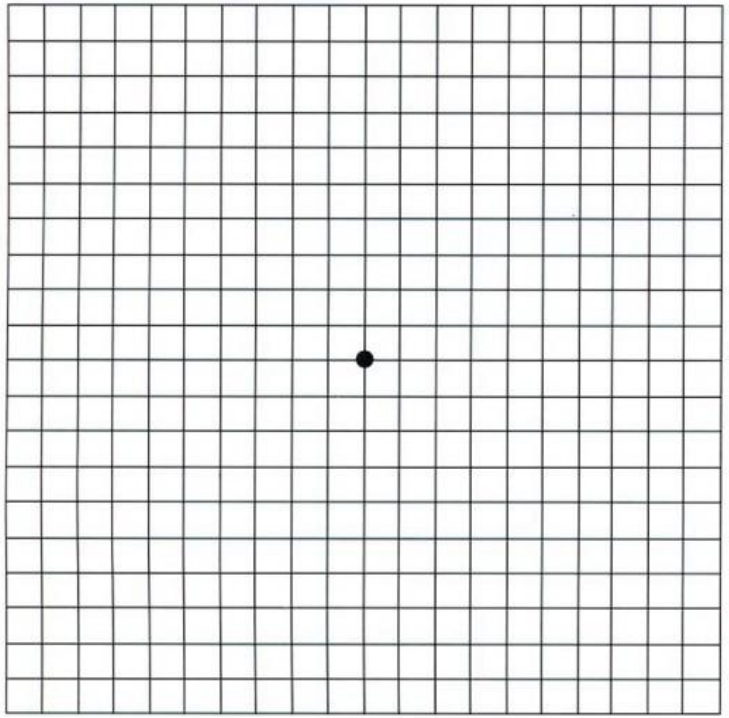
feature detection





pattern recognition

Matrix



rows: 長
cols: 寬
type: 像素型態
channels: 通道數

Coordinates

normal:

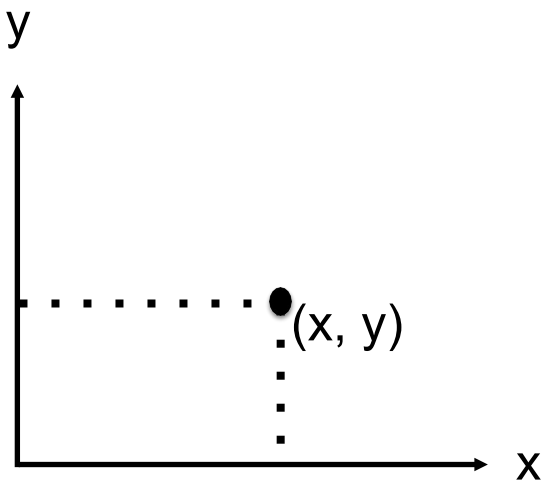


image:

```
0/0---X--->
 |
 |
 Y
 |
 |
 v
```

```
0/0---column--->
 |
 |
row
 |
 |
 v
```

Matrix value access

	Column 0	Column 1	Column ...	Column m
Row 0	0,0	0,1	...	0, m
Row 1	1,0	1,1	...	1, m
Row ...	...,0	...,1	...	..., m
Row n	n,0	n,1	n,...	n, m

3-channel : B, G, R

	Column 0			Column 1			Column ...			Column m		
Row 0	0,0	0,0	0,0	0,1	0,1	0,1	...	...	...	0, m	0, m	0, m
Row 1	1,0	1,0	1,0	1,1	1,1	1,1	...	...	...	1, m	1, m	1, m
Row ...	...,0	...,0	...,0	...,1	...,1	...,1	...	...	...	..., m	..., m	..., m
Row n	n,0	n,0	n,0	n,1	n,1	n,1	n,...	n,...	n,...	n, m	n, m	n, m

Example Code

```python
import numpy as np
import cv2

# read
image = cv2.imread("image.jpg")
# show
cv2.imshow("My Image", image)

# press key to close display window
cv2.waitKey(0)
cv2.destroyAllWindows()

# save
cv2.imwrite("output_image.jpg", image)
```

標頭引入

讀圖像

顯示圖片

等待按鍵輸入

存圖像

Example Code

- **開一個指定大小的黑圖：**

```
blank_image = np.zeros((height,width,3), np.uint8)
```

- **複製圖片：**

```
newImage = myImage.copy()
```

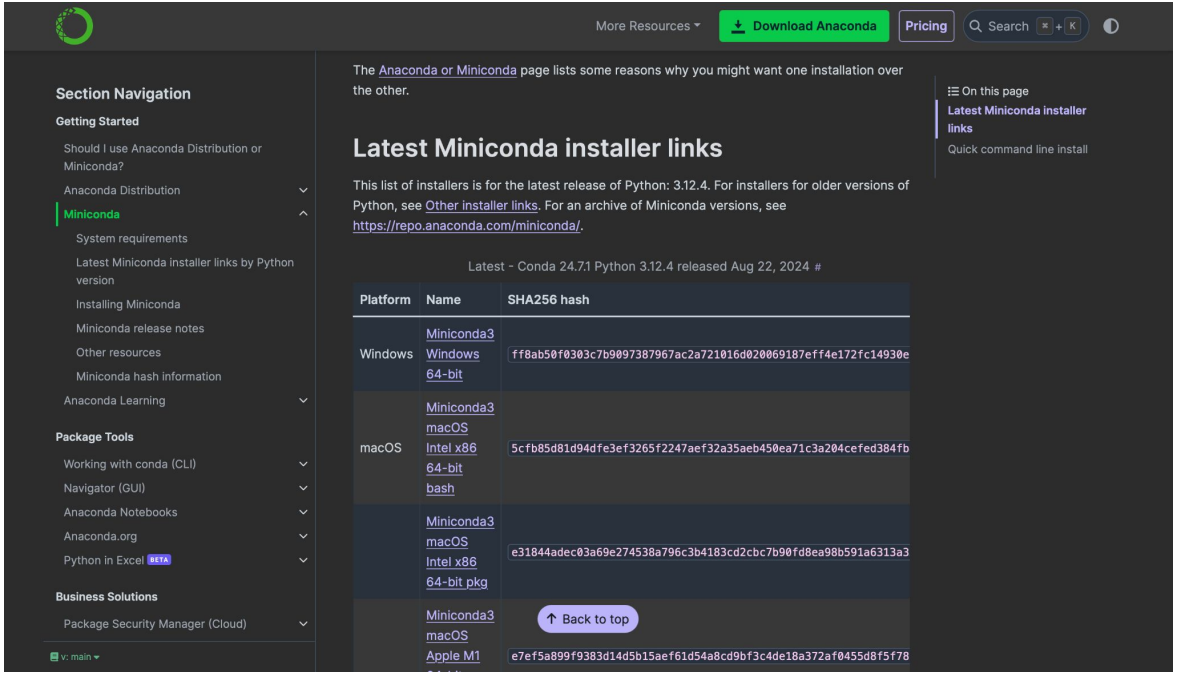
- **操作像素：**

image[row, col, channel]

Python 3x & OpenCV Installation

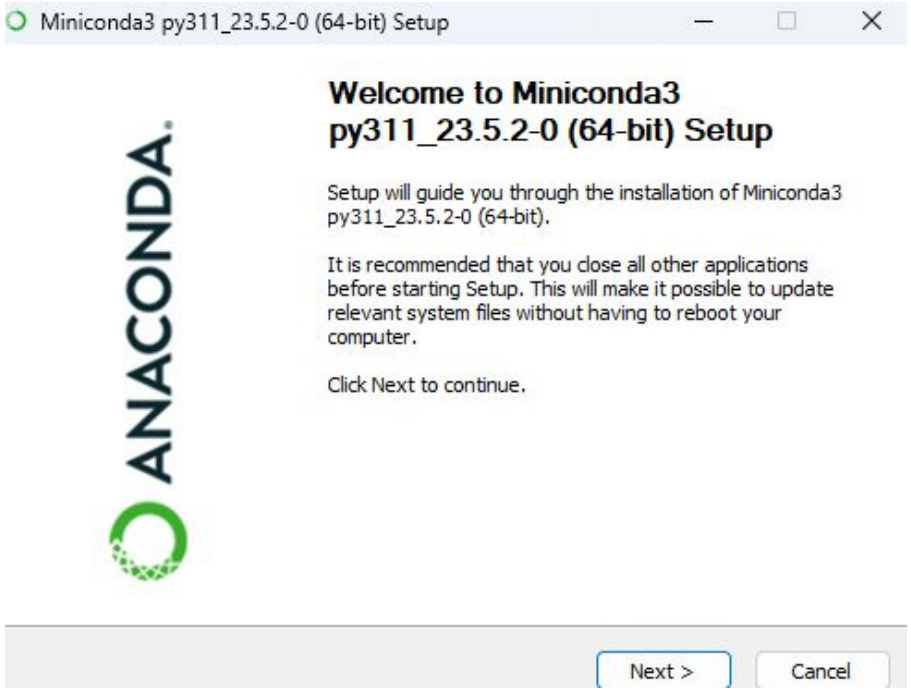
安装conda

- miniconda: https://docs.conda.io/projects/miniconda/en/latest/



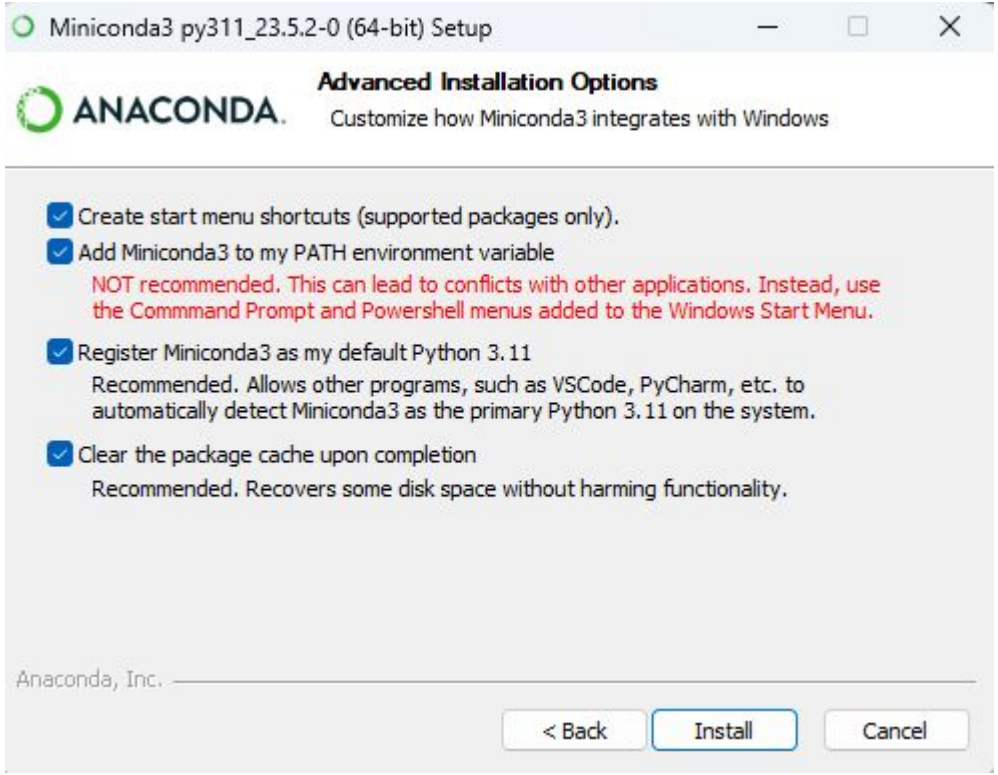
安裝conda

- 一直按next直到下一頁的畫面



安裝conda

- 這邊建議4個都打勾



安裝conda

- **conda activate**
  - 確認是否安裝成功
- **conda update conda**
  - 更新conda



創建conda環境

- conda create --name uav python=3.9

```
(base) C:\Users\covis223b>conda create --name uav python=3.9
```

- conda activate uav

```
(base) C:\Users\covis223b>conda activate uav

(uav) C:\Users\covis223b>
```

!!!務必使用conda環境操作無人機!!!

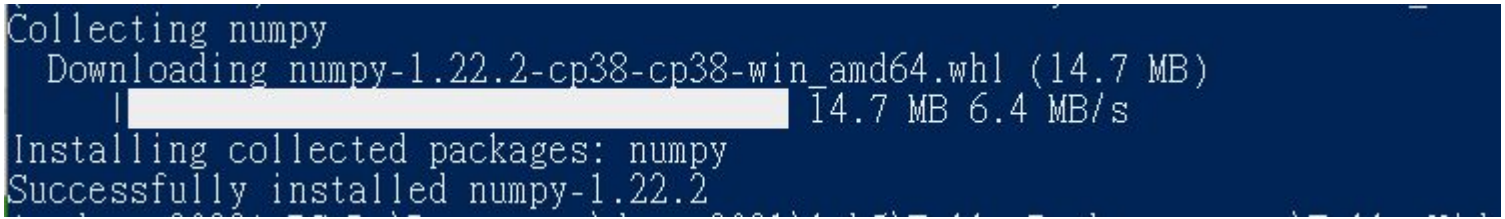
3. 安装opencv

- pip install opencv-python**==4.4.0.46**
- Test :

```
1  import cv2
2
3  img = cv2.imread('kobe.jpg')
4
5  cv2.imshow('My Image', img)
6  cv2.waitKey(0)
7  cv2.destroyAllWindows()
8
```

4. 安装numpy

- pip install numpy



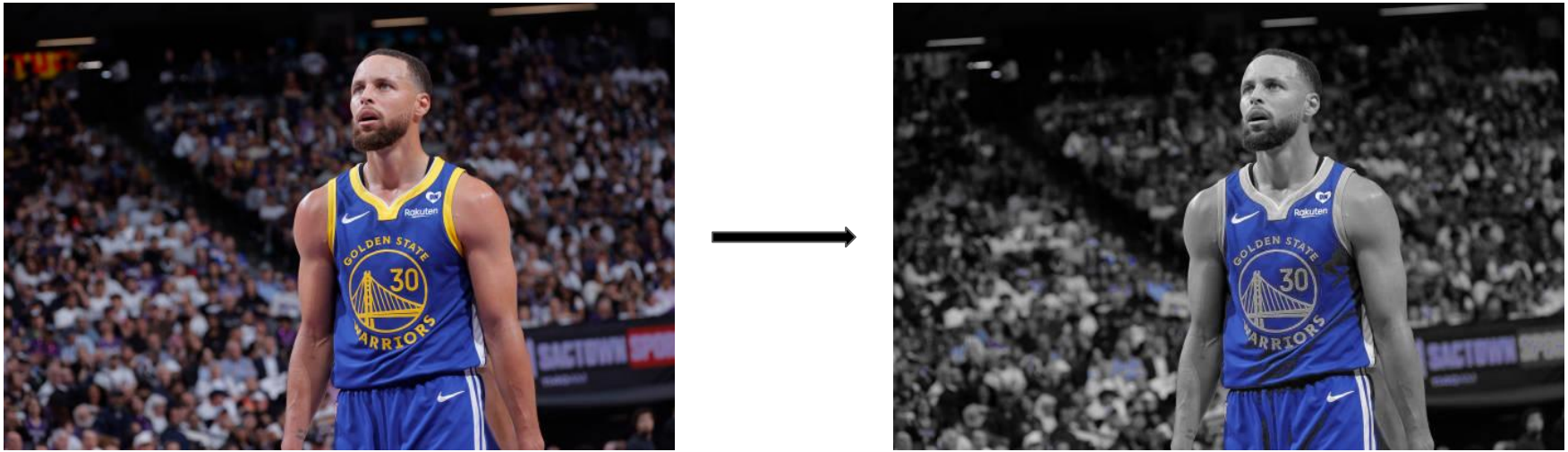
Lab1

Lab1

1. 圖片灰階與顏色濾鏡, 對比與亮度
2. Bilinear Interpolation
3. 邊緣偵測 (Filtering & Sobel Operator)

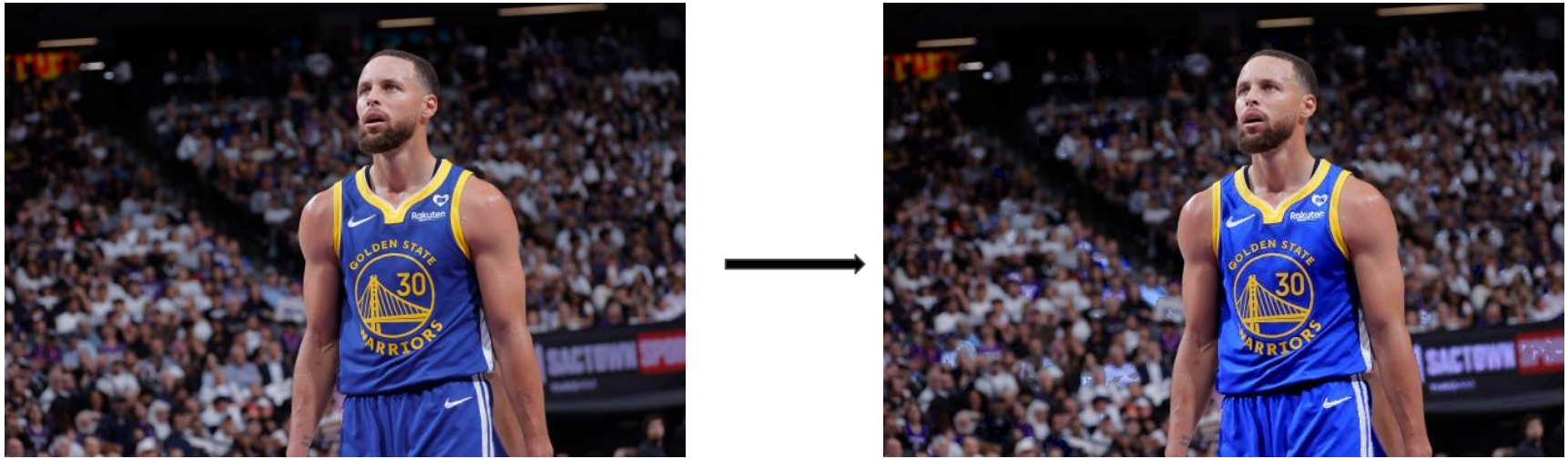
1.1 灰階與顏色濾鏡 (20%)

- 將原始圖片中的「藍點」予以保留，並把其餘的點改為灰階。
- Hint: B > 100 and B > 1.5 * R and B > 1.2 * G



1.2 對比與亮度 (20%)

- 更改原始圖片中的「藍點與黃點」像素的對比與亮度，其餘像素保持原樣



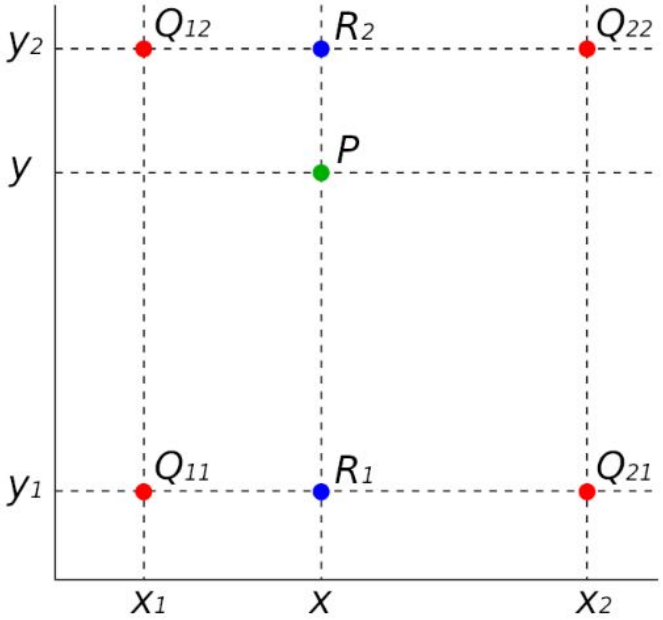
[Example] contrast=100, brightness=40

1.2 對比與亮度 (20%)

- 更改原始圖片中的「藍點與黃點」像素的對比與亮度，其餘像素保持原樣
  - Blue: (B > 1.5 * R) and (B > 1.2 * G)
  - Yellow: (R > 1.5 * B) and (G > 1.5 * B)
  - contrast: 50
  - brightness: 20
- new_image = (old_image - 127) × (contrast/127 + 1) + 127 + brightness
  - **Hint:** 記得注意overflow的問題
- 可能會用到的函式
  - np.array(img, dtype=np.int32), np.array(img, dtype=np.uint8)
  - np.clip(img, 0, 255)

2. Bilinear Interpolation (40%)

- 根據輸出影像的像素位置，找到輸入影像中最鄰近的四個點，再利用**雙線性內插法**求出輸出影像的像素強度



$$u = \frac{x - x_1}{x_2 - x_1}$$

$$w = \frac{y - y_1}{y_2 - y_1}$$

$$P = (1-u)(1-w)Q_{11} + u(1-w)Q_{21} + (1-u)wQ_{12} + uwQ_{22}$$

2. Bilinear Interpolation (40%)

- 將圖片放大3倍
- 評分標準
  - 使用 OpenCV API - **5 pts**
  - 自行實作雙線性內插法 - **40 pts**

3. 邊緣偵測 (Filtering & Sobel Operator) (30%)





3. 邊緣偵測 (Filtering & Sobel Operator) (30%)

- **Preprocessing** (可使用OpenCV內建function)
  - 將圖片轉為灰階
  - 對灰階圖做高斯模糊(去雜訊)
    - blur = cv2.GaussianBlur(gray, (5, 5), 1.0)
- **cv2.filter2D(img, -1, kernel)**

$$\mathbf{G_x} = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G_y} = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

$$G = \sqrt{G_x^2 + G_y^2}$$

Do NOT use cv2.Sobel() directly

Demo image for Q1 (q1.jpg)



Demo image for Q2 (q2.jpg)



Demo image for Q3 (q3.jpg)

